MACHINE LEARNING

Master in Data Science and Advanced Analytics

NOVA Information Management School

Universidade Nova de Lisboa

Handout

Machine Learning

Group 21

Mehmet Karaca, 20250344

Gonçalo Castro, 20250415

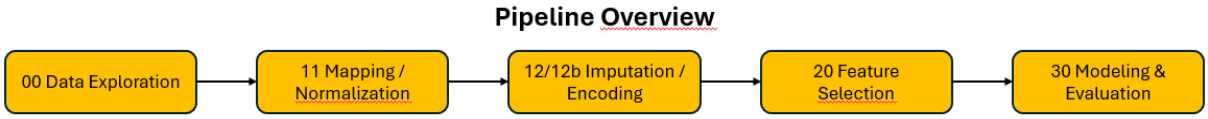
João Ramos, 20250413

Oliver Kain, 20250401

TABLE OF CONTENTS

PIPELINE OVERVIEW

We use an end-to-end pipeline: rule-based mapping/normalization (11) standardizes categories without using the target; on the training split we impute, encode, and engineer features (12/12b); we then run feature selection to remove redundancy and keep the most informative predictors (20). Finally, we train and tune the models, generate test predictions, and submit the results to Kaggle (30), optimizing for mean absolute error (MAE), the official competition metric.



DATA EXPLORATION

We work with a training set of 75,973 rows and 14 columns (including the target) and a test set of 32,567 rows and 12 columns. The main missing fields in the training data are *mpg* (10.43%) and *tax* (10.40%), with several others at roughly ~2%—*previousOwners*, *hasDamage*, *paintQuality%*, *transmission*, *Brand*, *model*, *engineSize*, *fuelType*, *year*, and *mileage*. We found no duplicates for *carID*. Before cleaning, we detected anomalies such as 358 cars with *year* > 2020, 2,214 rows with non-integer *year*, 2,284 rows with non-integer *previousOwners*, and 367 rows where *paintQuality%* exceeded 100. We also flagged unusual numeric patterns affecting *engineSize* (575 rows) and *mpg* (6,075 rows). Overall, the dataset is broadly usable but shows inconsistent types, outliers, and categorical noise, which makes targeted rule-based Normalization necessary before any statistical learning.

DATA MAPPING & NORMALIZATION

We standardize all text fields by lowercasing, trimming whitespace, and Normalizing punctuation, then harmonize categorical values through JSON alias and regex mappings for *transmission*, *fuelType*, *Brand*, and *model*. This deterministic mapping corrects misspellings and inconsistencies across train and test; for example, missing *Brand* entries drop from 1,521 to 116 in train and 649 to 42 in test after inferring *Brand* from *model* and applying consistent aliases. We also enforce numeric sanity by coercing impossible or inconsistent values to missing (NaN), such as negative *mileage*, *paintQuality%* outside the valid 0-100 range, or non-integer values for year and *previousOwners*. Additionally, the placeholder category “unknown” in *transmission* is treated as missing so it also can be imputed later within the pipeline. Because the entire step is purely rule-based and does not estimate statistics from the data, it is safe to perform before any split and does not introduce data leakage.

DATA IMPUTATION, ENCODING & FEATURE ENGINEERING

We preprocess all data using a train-only pipeline to avoid leakage. Missing numeric values are filled with medians, using structured logic for key fields: *mileage* by *year* medians, *year* by *mileage* bins, and *mpg*, *tax*, and *engineSize* by hierarchical medians across (*model*, *Brand*, *fuelType*). Categorical gaps (*transmission*, *fuelType*, *Brand*, *model*) are first filled with per-model modes when enough samples exist; remaining NAs are predicted with Random Forest classifiers trained on related features. Simpler fields (*paintQuality%*, *previousOwners*) use median fill. After imputation, low-cardinality categoricals are one-hot encoded (fit on train, handle_unknown="ignore"). We then engineer additional features to capture vehicle age, usage intensity, efficiency, and ownership effects. These include the car’s age in years, *mileage* normalized by age to reflect annual usage, fuel efficiency relative to *engineSize*, the deviation of a car’s fuel economy from the typical value for its *transmission* type, indicators for multiple

previous owners, and ratios linking engine size to tax level. Finally, all numeric columns are scaled with a RobustScaler fitted on training data and applied consistently to validation and test sets. We use RobustScaler because it is less sensitive to outliers than StandardScaler. A secondary baseline pipeline ("12b") performs the same steps but uses only global medians and modes for imputation, which empirically yields slightly better predictive results despite its simplicity.

FEATURE SELECTION

In Feature Selection, we first applied filter methods by dropping low-variance features (<0.01)—hasDamage, transmission_Other, fuelType_Electric, and fuelType_Other—and removing redundancy using pairwise Pearson, Spearman, and Kendall correlations, discarding a feature whenever at least two of the three coefficients exceeded 0.90. We then used wrapper selection with RFE (Linear, Ridge, and Lasso as base learners), scanning subset sizes and recording the best validation score per model. In parallel, we ran embedded selection with regularized linear models and inspected coefficient magnitudes as importance signals. Combining evidence from all three families, we finalized a feature set.

MODELING, EVALUATION, HYPERPARAMETER TUNING

In our baseline comparison on the validation split, **Random Forest** performed best with MAE 1549.51 (R^2 = 0.927), ahead of Decision Tree (MAE 2030.23; R^2 = 0.868), Ridge and Lasso (both around MAE 2908.6; R^2 = 0.786), Linear Regression (MAE 2908.69; R^2 = 0.786), and Elastic Net (MAE 2985.07; R^2 = 0.775). We then tuned a Random Forest via a small ParameterGrid and selected n_estimators=400, max_depth=40, max_features=0.5, min_samples_split=5, min_samples_leaf=1, which improved the validation MAE to 1549.36. For the final model, we refit the tuned Random Forest on the combined train+validation data and produced test predictions. We submit our predictions to the Kaggle competition and get a **MAE of 1555.80496** on the test Data.

DATA-LEAKAGE & REPRODUCIBILITY

We prevent data leakage by doing all the necessary steps we mentioned in the sections above like fitting all the transformers on the train set. The remaining risk is selection bias because both RFE and model tuning consult the same validation split; we will mitigate this by switching to RFECV (inner cross-validation) or by wrapping feature selection and the estimator in a single scikit-learn Pipeline and selecting models via cross-validated MAE. This will be the task for the second deliverable. For reproducibility, we fix random_state=42, save all intermediate processed/encoded datasets and the final submission, and document paths and configuration so the full run can be replicated end-to-end.

KEY RESULTS & NEXT STEPS

We obtained a strong baseline with a Random Forest model (validation MAE ≈ 1549). For the second Deliverable , we will replace the hold-out approach with cross-validated, leakage-safe feature selection and model tuning by wrapping preprocessing, RFECV, and training in a single scikit-learn Pipeline. For high-cardinality categories such as model and Brand, we will evaluate target-aware encodings and compare them with frequency encoding for robustness. We will also expand the hyperparameter search space, for example by increasing the number of estimators, allowing unlimited tree depth, and raising the minimum samples per leaf to prevent overly restrictive configurations. Finally, we will add systematic error analysis, including SHAP-based feature attributions and MAE breakdowns by price segments, to guide targeted improvements.